

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 1
Nom, prénom : COLLOMB Eliot		N° candidat : 02542564744
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 10 /03/2026
Organisation support de la réalisation professionnelle Réalisation sur le temps de la formation SIO.		
Intitulé de la réalisation professionnelle : Conception et développement d'une application web de gestion de tickets d'assistance informatique.		
Période de réalisation : 10/03 Lieu : Centre de formation, Rennes Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation ¹ (ressources fournies, résultats attendus) Dans le cadre de la formation BTS SIO option SLAM, cette réalisation avait pour objectif de concevoir et développer une application web permettant de gérer des demandes d'assistance informatique sous forme de tickets. L'application devait permettre : - la création de tickets par les utilisateurs - la consultation et le suivi des tickets - la modification du statut d'un ticket - la gestion des priorités - la gestion des utilisateurs et des rôles - la consultation d'un historique des demandes L'objectif était de produire une application fonctionnelle simulant un outil de support informatique utilisé en entreprise, tout en appliquant des pratiques professionnelles de développement (architecture claire, gestion des données, sécurité des accès). Le projet a été réalisé en suivant une démarche de conception incluant : - modélisation des données - définition des fonctionnalités - développement de l'interface utilisateur - implémentation de la logique métier - tests de fonctionnement		

¹ En référence aux conditions de réalisation et ressources nécessaires du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO. 6

Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées⁶

Ressources logicielles :

- Node.js
- Next.js
- TypeScript
- Prisma ORM
- SQLite
- Tailwind CSS
- Visual Studio Code
- Navigateur web (tests et débogage)

Ressources matérielles :

- Poste de développement sous Windows

Ressources documentaires :

- Documentation officielle : Next.js, Prisma ORM, TypeScript
- Tutoriels et documentation technique en ligne

Modalités d'accès aux productions² et à leur documentation³

L'application est accessible via une version déployée sur un serveur personnel virtualisé.

L'accès à l'application se fait à l'adresse suivante :

<http://eliot-ticketing.duckdns.org>

Le code source est également disponible à l'adresse suivante GitHub : <https://github.com/EliooolCb/ticketing.git>

En cas d'indisponibilité de l'accès distant, le projet peut être exécuté localement à partir du code source fourni (npm install / npm run dev).

Captures d'écran, fichiers et procédure documentée dans le dossier technique associé au projet.

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS

SESSION 2026

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle

(verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Contexte :

Dans les services informatiques, les demandes d'assistance sont généralement centralisées via un outil de ticketing. Ces outils permettent de structurer les demandes, suivre leur traitement et conserver un historique.

L'objectif du projet était donc de développer une application web simplifiée de gestion de tickets permettant de reproduire le fonctionnement d'un outil professionnel.

Analyse des besoins :

Gestion des tickets

- création d'un ticket
- consultation de la liste des tickets
- modification d'un ticket
- changement de statut
- gestion de la priorité

² Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

³ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

Gestion des utilisateurs

- identification de l'utilisateur
- attribution des tickets

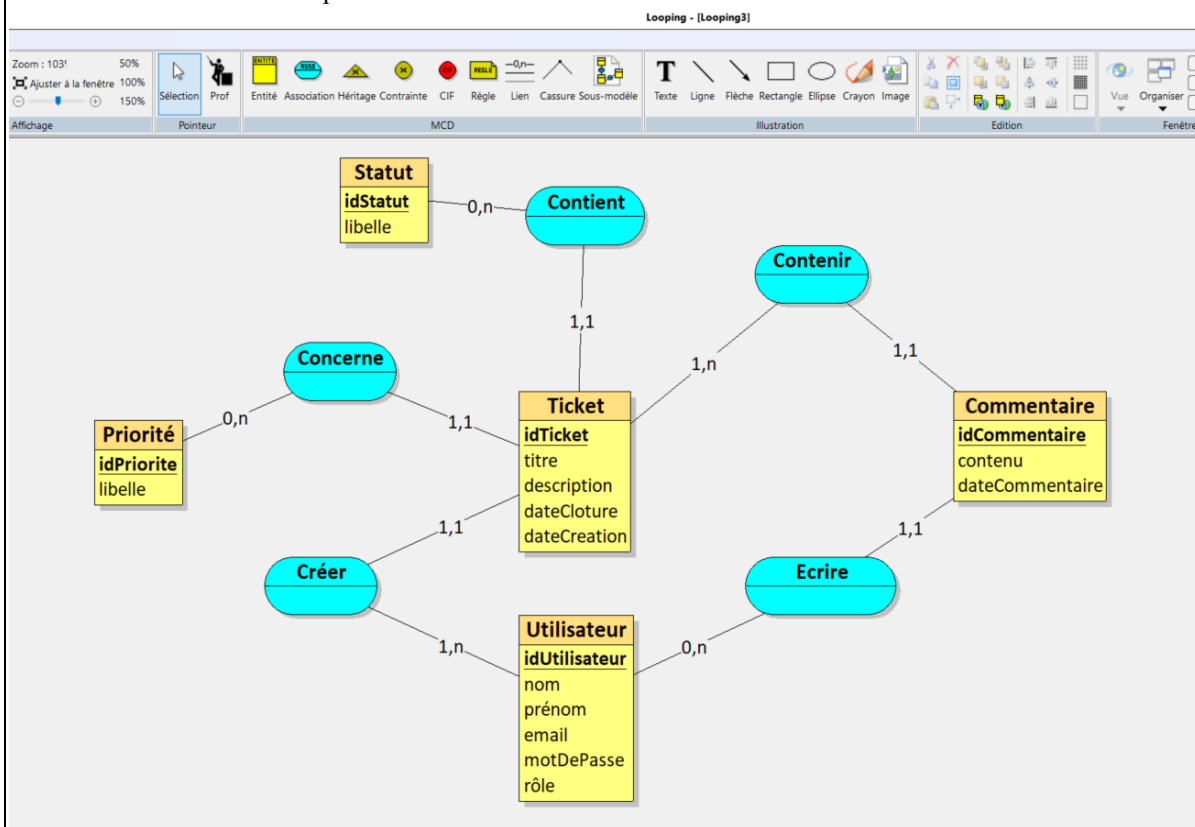
Suivi du traitement

- visualisation du statut du ticket
- suivi de l'évolution

Conception de la base de données

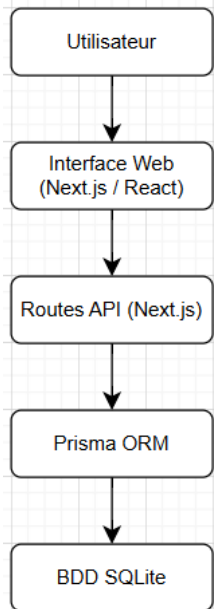
La structure des données a été modélisée avec le logiciel **Looping** afin de représenter les éléments principaux du système. Cette modélisation permet de structurer efficacement les informations nécessaires au suivi des demandes.

Screenshot du Modèle Conceptuel de Données :



Architecture de l'application :

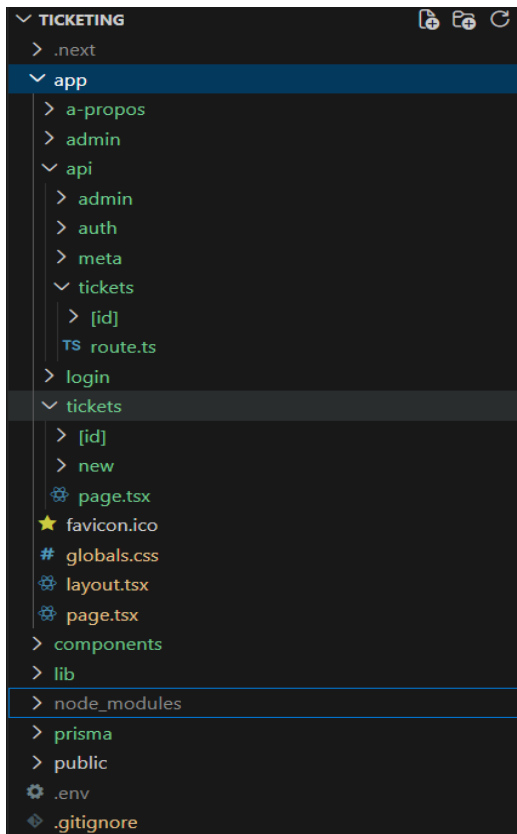
L'application a été développée sous la forme d'une application web full-stack, combinant une interface utilisateur et une couche backend permettant de gérer la logique métier et l'accès aux données. Cette architecture permet de mettre en place une séparation claire entre le frontend, le backend et la gestion des données, facilitant la maintenance et l'évolution de l'application.



La partie **frontend** repose sur le framework **Next.js**, qui permet de développer des interfaces web modernes basées sur la bibliothèque **React**. L'utilisation de **TypeScript** permet d'améliorer la robustesse du code grâce au typage statique et facilite la maintenance de l'application. La mise en forme de l'interface utilisateur est assurée par le framework **CSS Tailwind**, qui permet de structurer rapidement les composants visuels.

La partie **backend** est implémentée à l'aide des API routes fournies par **Next.js**. Cette approche permet de centraliser les opérations liées à la gestion des données et d'implémenter la logique métier de l'application.

L'accès à la base de données est réalisé à l'aide de **Prisma ORM**, qui facilite l'interaction avec la base de données et permet de manipuler les données sous forme d'objets plutôt que de requêtes SQL brutes.



La capture ci-contre présente l'organisation des fichiers du projet dans l'environnement de développement Visual Studio Code.

L'application est structurée en plusieurs répertoires afin de séparer les différentes responsabilités de l'application.

Le dossier **app** contient les pages et les routes de l'application web, tandis que le dossier **api** regroupe les endpoints permettant de gérer les opérations liées aux tickets.

Le dossier **prisma** contient la définition du schéma de base de données et permet de gérer les interactions avec la base SQLite via l'ORM Prisma.

Cette organisation permet de structurer le projet et facilite la maintenance ainsi que l'évolution de l'application.

Gestion de données :

La base de données utilisée pour ce projet est une base SQLite, adaptée à un environnement de développement local et permettant de simplifier la configuration du projet.

Le modèle de données est défini dans le schéma Prisma, qui permet de décrire les entités du système ainsi que leurs relations. Prisma génère ensuite automatiquement un client permettant d'effectuer des opérations sur la base de données.

Extrait de schema.prisma pour l'entité Utilisateur :

```
15 model Utilisateur {
16   idUtilisateur Int    @id @default(autoincrement())
17   nom            String
18   prenom        String
19   email         String @unique
20   motDePasse    String
21   role          Role
22
23   // 1 utilisateur -> 1..n tickets créés (contrainte métier contrôlée à la création)
24   ticketsCrees Ticket[] @relation("UtilisateurTickets")
25   // 1 utilisateur -> 0..n commentaires
26   commentaires Commentaire[] @relation("UtilisateurCommentaires")
27 }
```

Les principales opérations réalisées sur les données correspondent aux opérations classiques de gestion des informations, notamment :

- la création de nouveaux tickets
- la consultation de la liste des tickets
- la modification d'un ticket existant
- la suppression d'un ticket

Ces opérations sont implémentées dans les routes API de l'application et permettent de gérer l'ensemble du cycle de vie des tickets.

Extrait d'une route API – création d'un ticket :

```

export async function POST(request: Request): Promise<NextResponse> {
  const user = await requireUser();
  const body = await request.json();
  const parsed = ticketCreateSchema.safeParse(body);

  if (!parsed.success) {
    return fail(400, "Données ticket invalides");
  }

  const ticket = await prisma.ticket.create({
    data: {
      titre: parsed.data.titre,
      description: parsed.data.description,
      idStatut: parsed.data.idStatut,
      idPriorite: parsed.data.idPriorite,
      idCreateur: user.idUtilisateur,
      commentaires: {
        create: {
          contenu: `Création du ticket : ${parsed.data.description}`,
          idAuteur: user.idUtilisateur,
        },
      },
    },
  });

  return ok(ticket, { status: 201 });
}

```

Cet extrait présente l'implémentation d'une route API permettant la création d'un ticket.

Les données envoyées par le formulaire sont récupérées, validées puis enregistrées en base de données à l'aide de Prisma ORM.

Un premier commentaire est automatiquement créé lors de l'enregistrement afin de respecter la logique métier définie lors de la conception.

Mise en place d'un accès sécurisé (version démo) :

Une interface d'authentification a été intégrée à l'application afin de simuler un accès sécurisé aux fonctionnalités.

Lors de la première connexion, l'utilisateur est redirigé vers une page de connexion permettant de saisir ses identifiants.

Afin de faciliter les tests par le jury, des comptes de démonstration ont été mis en place, permettant notamment de tester les différentes fonctionnalités. Cette approche permet de reproduire un fonctionnement proche d'une application professionnelle, tout en garantissant une utilisation simple lors de la démonstration.

Connexion

Connecte-toi avec un compte de démonstration pour tester l'application.

Email

Mot de passe

Se connecter

Comptes de démonstration (seed)

- admin@ticketing.local / admin123
- user@ticketing.local / user123

Développement de l'interface utilisateur :

L'interface utilisateur a été conçue afin de proposer une navigation simple et intuitive.

La page principale de l'application permet d'afficher la liste des tickets enregistrés dans le système. Cette vue offre une vision globale des demandes d'assistance et permet d'accéder rapidement au détail d'un ticket.

← → ↺

localhost:3000/tickets

🔍 ☆ E ⋮

Ticketing IT

Tickets Nouveau Aide Référentiels

Eliot Admin (admin) Se déconnecter

Tickets

Créer un ticket

Tous statuts ▼

Toutes priorités ▼

Tous créateurs ▼

Ouverts + clos ▼

Tri: plus récents ▼

Filtrer

Réinitialiser

Titre	Statut	Priorité	Créateur	Création	Commentaires
#4 - Test 4 User	En attente	Critique	Bob User	2 mars 2026, 17:29	1
#3 - Un problème 3 Mes tickets	En attente	Moyenne	Eliot Admin	2 mars 2026, 17:09	1
#2 - Test 2 ticket user	Résolu	Haute	Bob User	2 mars 2026, 17:02	1
#1 - Test 1 Mes tickets	Résolu	Moyenne	Eliot Admin	2 mars 2026, 17:00	3

Un formulaire de création permet à l'utilisateur de saisir les informations nécessaires à l'ouverture d'une nouvelle demande. Les données saisies sont ensuite envoyées au backend de l'application afin d'être enregistrées dans la base de données.

Ticketing IT

Tickets Nouveau Aide Référentiels

Eliot Admin (admin) Se déconnecter

Nouveau ticket

Titre

Description

Statut

En attente ▼

Priorité

Moyenne ▼

Créer le ticket

Chaque ticket peut également être consulté individuellement, ce qui permet d'afficher l'ensemble des informations associées à la demande et d'effectuer d'éventuelles modifications.

Ticket #2 - Test 2 ticket user

Clôturer

Créateur : Bob User

Date création : 2 mars 2026, 17:02

Statut : Résolu

Priorité : Haute

Date clôture : -

Modifier le ticket

Test 2 ticket user

Bonjour j'ai un problème je suis un user désespéré

Résolu

Haute

Enregistrer

Commentaires (ordre chronologique)

Ajouter un commentaire...

Ajouter le commentaire

Extrait du composant de création de ticket (ticket-create-form.tsx) :

```
async function handleSubmit(event: FormEvent<HTMLFormElement>): Promise<void> {
  event.preventDefault();
  setError("");
  setLoading(true);

  try {
    const response = await fetch("/api/tickets", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ titre, description, idStatut, idPriorite }),
    });

    const data = await response.json();

    if (!response.ok) {
      setError(data.error ?? "Erreur de création");
      return;
    }

    router.push(`/tickets/${data.idTicket}`);
  } catch {
    setError("Erreur réseau pendant la création du ticket.");
  } finally {
    setLoading(false);
  }
}
```

Cet extrait présente le composant React permettant la création d'un ticket.

Il gère la saisie des informations (titre, description, statut, priorité), ainsi que l'envoi des données vers l'API via une requête HTTP POST.

Une gestion des erreurs et de l'état de chargement est également mise en place afin d'améliorer l'expérience utilisateur.

Tests et validation :

Plusieurs tests fonctionnels ont été réalisés afin de vérifier le bon fonctionnement de l'application. Ces tests ont notamment permis de valider la création des tickets, leur enregistrement dans la base de données ainsi que leur affichage dans l'interface utilisateur.

Des tests ont également été réalisés afin de vérifier la modification des informations d'un ticket et la cohérence des données affichées après chaque opération.

Ces différentes vérifications ont permis de confirmer que l'application répond correctement aux fonctionnalités définies lors de l'analyse des besoins.

Résultats obtenus :

La réalisation de ce projet a permis de développer une application web fonctionnelle de gestion de tickets d'assistance informatique. L'application permet de centraliser les demandes des utilisateurs, d'en assurer le suivi et de structurer les informations associées à chaque ticket.

Ce projet illustre la mise en œuvre d'une solution applicative complète, intégrant la conception du modèle de données, le développement d'une interface utilisateur et la gestion des interactions avec la base de données.

La réalisation mobilise ainsi plusieurs compétences du référentiel BTS SIO, notamment la conception et le développement d'une solution applicative, la gestion des données ainsi que la maintenance et l'évolution d'une application.

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS SESSION 2026
 ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (recto)
 Épreuve E6 - Conception et développement d'applications (option SLAM)

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 2
Nom, prénom : COLLOMB Eliot		N° candidat : 02542564744
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 10 /03/2026
Organisation support de la réalisation professionnelle Réalisation sur le temps de la formation SIO.		
Intitulé de la réalisation professionnelle : Conception et développement d'un système d'authentification sécurisée avec gestion des rôles dans l'application web Mycellius.		
Période de réalisation : 10/03 Lieu : Centre de formation, Rennes Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation ⁴ (ressources fournies, résultats attendus) Dans le cadre de la formation BTS SIO option SLAM, cette réalisation avait pour objectif de sécuriser l'accès à l'application Mycellius, un wiki interne d'entreprise accessible depuis une interface web. L'objectif était de mettre en place une authentification moderne et sécurisée reposant sur des jetons JWT, ainsi qu'un contrôle d'accès fondé sur les rôles utilisateurs. L'application devait permettre : - l'authentification d'un utilisateur via un formulaire de connexion - la vérification des identifiants côté serveur - la génération d'un token JWT après connexion - la sécurisation des endpoints de l'API - la gestion des rôles STAGIAIRE, DEV et ADMIN - la restriction de certaines actions selon le rôle - la gestion côté interface des erreurs d'accès et des sessions utilisateur L'objectif était de produire une fonctionnalité de sécurité réaliste, proche de ce que l'on retrouve dans une application professionnelle, tout en appliquant des pratiques de développement sécurisées : hashage des mots de passe, contrôle d'accès, séparation front/back et gestion centralisée de l'authentification.		

⁴ En référence aux conditions de réalisation et ressources nécessaires du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO. 6

Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées⁶

Ressources logicielles :

Java 21
Spring Boot
Spring Security
JWT
React
Vite
MySQL
Flyway
Maven
IntelliJ IDEA / Visual Studio Code
Postman
Docker / Docker Compose

Ressources matérielles :

Poste de développement sous Windows
Environnement local de test

Ressources documentaires :

Documentation officielle Spring Boot
Documentation Spring Security
Documentation React / Vite
Documentation MySQL / Flyway
Supports de TP Mycellius

Modalités d'accès aux productions⁵ et à leur documentation⁶

L'application Mycellius est accessible via une interface web reliée à une API Spring Boot.

Elle est accessible à l'adresse suivante :

<http://eliot-mycellius.duckdns.org>

Le lien GitHub est également disponible via le lien associé : <https://github.com/ElloooClb/mycellius-sio-eliot.git>

Les productions sont consultables à travers le code source, les captures d'écran, les tests Postman ainsi que la documentation technique associée.

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS

SESSION 2026

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

Contexte :

Dans une application web d'entreprise, la sécurisation des accès constitue une étape essentielle afin de protéger les données et de limiter les actions possibles selon le profil utilisateur.

Le projet Mycellius étant conçu comme un wiki interne d'entreprise, il était nécessaire de mettre en place une authentification permettant d'identifier les utilisateurs et de contrôler leurs droits d'accès.

⁵ Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁶ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

L'objectif de cette réalisation était donc de développer un système complet de connexion sécurisée, associé à une gestion des rôles, afin de distinguer les utilisateurs autorisés à consulter les contenus de ceux pouvant les créer, modifier ou administrer.

Analyse des besoins :

Authentification :

- saisie d'un identifiant et d'un mot de passe
- vérification des identifiants côté serveur
- retour d'un token JWT en cas de succès

Sécurisation des accès :

- protection des routes de l'API
- refus des requêtes non authentifiées
- contrôle des actions selon le rôle utilisateur

Gestion des rôles :

- STAGIAIRE : consultation uniquement
- DEV : consultation, création et modification
- ADMIN : accès complet

Interface utilisateur :

- formulaire de connexion
- conservation du token côté client
- redirection si non connecté
- affichage d'une erreur en cas d'échec d'authentification ou d'accès interdit

Conception de la solution :

Modélisation des données de sécurité

La gestion des accès repose sur une table utilisateur stockée en base de données. Chaque utilisateur possède un nom de connexion, un mot de passe stocké sous forme hashée ainsi qu'un rôle permettant de déterminer ses droits d'accès dans l'application.

La structure de cette table a été mise en place via une migration Flyway, ce qui permet de versionner le schéma de base de données et de garantir un déploiement reproductible.

Création de la table utilisateur via Flyway :

```
api > src > main > resources > db > migration > V5__create_app_user_table.sql
1 CREATE TABLE IF NOT EXISTS app_user (
2     id BIGINT NOT NULL AUTO_INCREMENT,
3     username VARCHAR(80) NOT NULL,
4     password_hash VARCHAR(255) NOT NULL,
5     role VARCHAR(20) NOT NULL,
6     PRIMARY KEY (id),
7     UNIQUE KEY uk_app_user_username (username)
8 ) ENGINE=InnoDB;
```

Cet extrait présente la migration SQL utilisée pour créer la table app_user.

Cette table permet de stocker les informations des utilisateurs, notamment leur identifiant, leur mot de passe hashé ainsi que leur rôle.

L'utilisation de Flyway permet de versionner la structure de la base de données et de garantir un déploiement cohérent.

Le mapping objet-relationnel a ensuite été réalisé à l'aide de JPA, avec une entité UserEntity représentant la table app_user et un repository permettant de rechercher un utilisateur par son identifiant de connexion.

Mapping JPA de l'entité utilisateur :

```
api > src > main > java > fr > mycellius > persistence > entity > J UserEntity.java > UserEntity
1  package fr.mycellius.persistence.entity;
2  import jakarta.persistence.*;
3  @Entity
4  @Table(name = "app_user")
5  public class UserEntity {
6      @Id
7      @GeneratedValue(strategy = GenerationType.IDENTITY)
8      private Long id;
9      @Column(nullable = false, unique = true, length = 80)
10     private String username;
11     @Column(name = "password_hash", nullable = false, length = 255)
12     private String passwordHash;
13     @Column(nullable = false, length = 20)
14     private String role; // ADMIN / DEV / STAGIAIRE
```

Accès aux utilisateurs via JPA Repository :

```
api > src > main > java > fr > mycellius > persistence > jpa > J UserJpaRepository.java > UserJpaRepository
1  package fr.mycellius.persistence.jpa;
2  import fr.mycellius.persistence.entity.UserEntity;
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import java.util.Optional;
5  public interface UserJpaRepository extends JpaRepository<UserEntity, Long> {
6      Optional<UserEntity> findByUsername(String username);
7  }
```

Cet extrait présente le repository JPA utilisé pour accéder aux données utilisateur.

Il permet notamment de rechercher un utilisateur à partir de son nom de connexion lors de la phase d'authentification.

L'extension de JpaRepository fournit automatiquement les opérations de base sur la base de données. Ce repository est utilisé lors de l'authentification afin de récupérer l'utilisateur en base et de vérifier ses identifiants.

Architecture de l'application

L'application Mycellius repose sur une architecture séparant clairement le frontend et le backend.

La partie frontend a été développée avec React et Vite. Elle gère l'affichage du formulaire de connexion, l'envoi des identifiants à l'API, le stockage du token JWT dans le navigateur ainsi que la navigation entre les pages selon l'état d'authentification.

La partie backend repose sur Spring Boot et Spring Security. Elle assure la vérification des identifiants, la génération des jetons JWT, la protection des routes de l'API ainsi que le contrôle d'accès en fonction des rôles utilisateurs.

La persistance des données est assurée par MySQL. Le schéma est versionné grâce à Flyway, ce qui permet de conserver une structure cohérente et de faire évoluer la base de manière contrôlée.

Cette architecture permet de séparer les responsabilités entre l'interface utilisateur, la logique métier de sécurité et la couche de persistance, ce qui facilite la maintenance et l'évolution de la solution.

1. l'utilisateur saisit ses identifiants
2. le front envoie une requête POST `/api/v1/auth/login`
3. l'API vérifie le mot de passe
4. l'API renvoie un JWT
5. le front stocke le token
6. les requêtes suivantes utilisent `Authorization: Bearer ...`
7. le filtre JWT valide le token et Spring Security applique les droits

Gestion des données :

La base de données contient les utilisateurs de l'application, avec leurs rôles et leurs mots de passe stockés sous forme hashée.

Une migration Flyway a été utilisée pour créer la table `app_user`, garantissant un déploiement structuré et reproductible du schéma.

L'accès aux données est réalisé via JPA, avec une entité utilisateur et un repository permettant de rechercher un utilisateur à partir de son nom de connexion.

Cette organisation permet de séparer la logique métier de la couche de persistance, tout en facilitant l'évolution future du système d'authentification.

Développement de l'interface utilisateur :

L'interface utilisateur comprend une page de connexion développée en React. Cette page permet à l'utilisateur de saisir ses identifiants puis d'envoyer une requête d'authentification à l'API.

Après validation, le token JWT, le nom d'utilisateur et le rôle sont stockés dans le navigateur afin de maintenir l'état de connexion.

Un contexte d'authentification a été mis en place pour centraliser ces informations dans l'application et permettre l'accès aux différentes pages en fonction de l'état de session.

Des redirections automatiques sont prévues lorsque l'utilisateur n'est pas authentifié ou lorsqu'il tente d'accéder à une fonctionnalité interdite.

Lors de la connexion, une requête POST est envoyée à l'API pour vérifier les identifiants.

Gestion centralisée de l'authentification côté client :

```
const [token, setToken] = useState(localStorage.getItem("token"));
const [role, setRole] = useState(localStorage.getItem("role"));
const [username, setUsername] = useState(localStorage.getItem("username"));
const value = useMemo(() => ({
  token,
  role,
  username,
  login: ({ token, role, username }) => {
    localStorage.setItem("token", token);
    localStorage.setItem("role", role);
    localStorage.setItem("username", username);
    setToken(token);
    setRole(role);
    setUsername(username);
  },
  logout: () => {
    localStorage.removeItem("token");
    localStorage.removeItem("role");
    localStorage.removeItem("username");
    setToken(null);
    setRole(null);
    setUsername(null);
  }
}), [token, role, username]);
```

Cet extrait présente le contexte d'authentification utilisé pour gérer la session utilisateur côté client.

Il permet de stocker le token JWT, le rôle et le nom d'utilisateur dans le navigateur, et de centraliser les fonctions de connexion et de déconnexion.

Cette approche facilite la gestion de l'accès aux différentes pages de l'application.

Les requêtes envoyées à l'API utilisent ensuite ce token pour accéder aux routes protégées (voir Figure suivante).

Communication entre le frontend et l'API sécurisée :

```
export async function apiRequest(path, { method = "GET", token = null, body = null } = {}) {
  const headers = { "Content-Type": "application/json" };

  if (path === "/api/v1/auth/login" && method === "GET") {
    throw new Error("apiRequest: /api/v1/auth/login doit être appelé en POST (pas en GET).");
  }

  const effectiveToken = token || localStorage.getItem("token");
  if (effectiveToken) headers.Authorization = `Bearer ${effectiveToken}`;

  const res = await fetch(`${API_URL}${path}`, {
    method,
    headers,
    body: body ? JSON.stringify(body) : null,
  });

  const text = await res.text();
  let data = null;
  try {
    data = text ? JSON.parse(text) : null;
  } catch {
    data = text;
  }

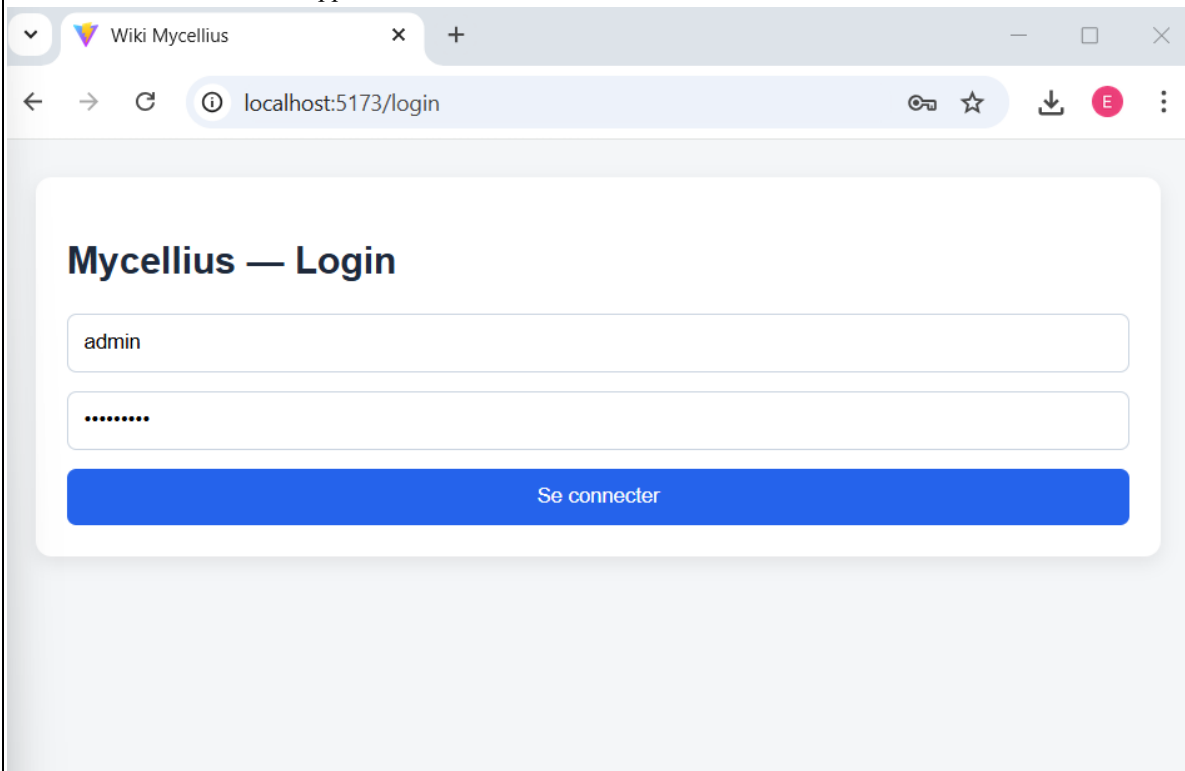
  if (!res.ok) throw new ApiError(res.status, data);
  return data;
}
```

Cet extrait présente la fonction utilisée pour envoyer les requêtes HTTP du frontend vers l'API.

Le token JWT est automatiquement récupéré puis ajouté dans l'en-tête Authorization, ce qui permet d'accéder aux routes protégées de l'application.

La fonction centralise également l'envoi des données et la gestion des erreurs de réponse.

Interface de connexion de l'application :



The screenshot shows a web browser window with the title "Wiki Mycellius". The address bar displays "localhost:5173/login". The main content area features a login form titled "Mycellius — Login". The form contains two input fields: the first is labeled "admin" and the second is masked with dots. Below these fields is a blue button labeled "Se connecter".

Sécurisation de la solution :

La sécurité de la solution repose sur plusieurs mécanismes complémentaires :

- les mots de passe ne sont pas stockés en clair mais hashés avec BCrypt
- l'authentification repose sur des jetons JWT signés
- les endpoints sensibles de l'API sont protégés par Spring Security
- un filtre JWT vérifie le token à chaque requête
- les droits sont gérés selon le rôle de l'utilisateur
- le front gère les erreurs 401 et 403 afin d'adapter l'interface au niveau de l'autorisation

Génération du jeton JWT :

```
public String generateToken(String username, String role) {
    Instant now = Instant.now();
    Instant exp = now.plusSeconds(ttlSeconds);
    return JWT.create()
        .withIssuer(issuer)
        .withSubject(username)
        .withClaim("role", role)
        .withIssuedAt(Date.from(now))
        .withExpiresAt(Date.from(exp))
        .sign(algo);
}

public DecodedJWT verify(String token) {
    return verifier.verify(token);
}
```

Vérification du JWT à chaque requête :

```
if (header != null && header.startsWith(prefix: "Bearer ")) {
    String token = header.substring(beginIndex: 7);
    try {
        DecodedJWT jwt = jwtService.verify(token);
        String username = jwt.getSubject();
        String role = jwt.getClaim("role").asString();

        if (uri.startsWith(prefix: "/api/v1/pages")) {
            System.out.println("[JWT] OK user=" + username + " role=" + role);
        }

        var auth = new UsernamePasswordAuthenticationToken(
            username,
            null,
            List.of(new SimpleGrantedAuthority("ROLE_" + role))
        );
        SecurityContextHolder.getContext().setAuthentication(auth);
    } catch (Exception ex) {
        System.err.println("[JWT] INVALID on " + uri + " -> " + ex.getClass().getSimpleName() + ": " + ex.getMessage());
        ex.printStackTrace();
        SecurityContextHolder.clearContext();
    }
}

filterChain.doFilter(request, response);
```

Cet extrait présente le filtre JWT chargé d'intercepter les requêtes protégées.

Il récupère le token transmis dans l'en-tête Authorization, vérifie sa validité, extrait le nom d'utilisateur et le rôle, puis place l'utilisateur authentifié dans le contexte de sécurité Spring.

Ce mécanisme permet de sécuriser l'accès aux routes de l'API sans utiliser de session serveur.

Configuration des droits d'accès par rôle avec Spring Security :

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.OPTIONS, ...patterns: "/*").permitAll()
    .requestMatchers(...patterns: "/error").permitAll()
    .requestMatchers(...patterns: "/api/v1/auth/login").permitAll()
    .requestMatchers(...patterns: "/api/health", "/actuator/health").permitAll()
    .requestMatchers(...patterns: "/swagger-ui/**", "/v3/api-docs/**").permitAll()
    // RBAC Pages
    // RBAC Pages
    .requestMatchers(HttpMethod.GET, ...patterns: "/api/v1/pages", "/api/v1/pages/**")
    .hasAnyRole(...roles: "STAGIAIRE", "DEV", "ADMIN")
    .requestMatchers(HttpMethod.POST, ...patterns: "/api/v1/pages", "/api/v1/pages/**")
    .hasAnyRole(...roles: "DEV", "ADMIN")
    .requestMatchers(HttpMethod.DELETE, ...patterns: "/api/v1/pages/**")
    .hasAnyRole(...roles: "DEV", "ADMIN")
    .requestMatchers(HttpMethod.PUT, ...patterns: "/api/v1/pages", "/api/v1/pages/**")
    .hasAnyRole(...roles: "DEV", "ADMIN")
    .anyRequest().authenticated()
)
.addFilterBefore(new JwtAuthenticationFilter(jwtService),
    beforeFilter: UsernamePasswordAuthenticationFilter.class);
```

Cet extrait présente la configuration de sécurité de l'application avec Spring Security.

Il permet de définir les routes publiques, de protéger les endpoints sensibles et d'appliquer un contrôle d'accès selon les rôles STAGIAIRE, DEV et ADMIN.

Un filtre JWT est également ajouté afin de vérifier le token transmis à chaque requête.

Tests et validation :

Tests API réalisés avec Postman :

- authentification avec identifiants valides ;
- authentification avec identifiants invalides ;
- accès à une route protégée sans token ;
- accès avec token valide ;
- tentative d'action interdite selon le rôle.

Tests frontend réalisés sur l'interface :

- affichage du formulaire de connexion ;
- connexion réussie avec redirection ;
- déconnexion ;
- affichage d'un message d'erreur en cas d'échec ;
- redirection automatique vers la page de connexion en l'absence de token.

Ces tests ont permis de valider la cohérence du fonctionnement entre le frontend, le backend et la gestion des rôles.

Conclusion :

Cette réalisation a permis de mettre en place un système d'authentification sécurisé et fonctionnel au sein de l'application Mycellius.

La solution développée permet d'identifier les utilisateurs, de sécuriser l'accès aux ressources de l'API et de différencier les droits selon les profils.

Cette réalisation a mobilisé des compétences en développement full-stack, en gestion des données et en sécurité applicative. Elle constitue une étape importante dans la professionnalisation du projet, en rapprochant l'application d'un fonctionnement réaliste en contexte d'entreprise.